AYURLEX

AI Research Agent — Blueprint

Multilingual, RAG-based IP & Regulatory Assistant for Ayurveda

Table of Contents

1. Vision
2. High-Level Architecture
3. Core Modules
4. Tech Stack (Suggested)
5. Data Pipeline
6. Staged Build Roadmap
7. Evaluation Framework
8. Key Risks to Design Against
9. Implementation Architecture — Frontend & Backend

1. Vision

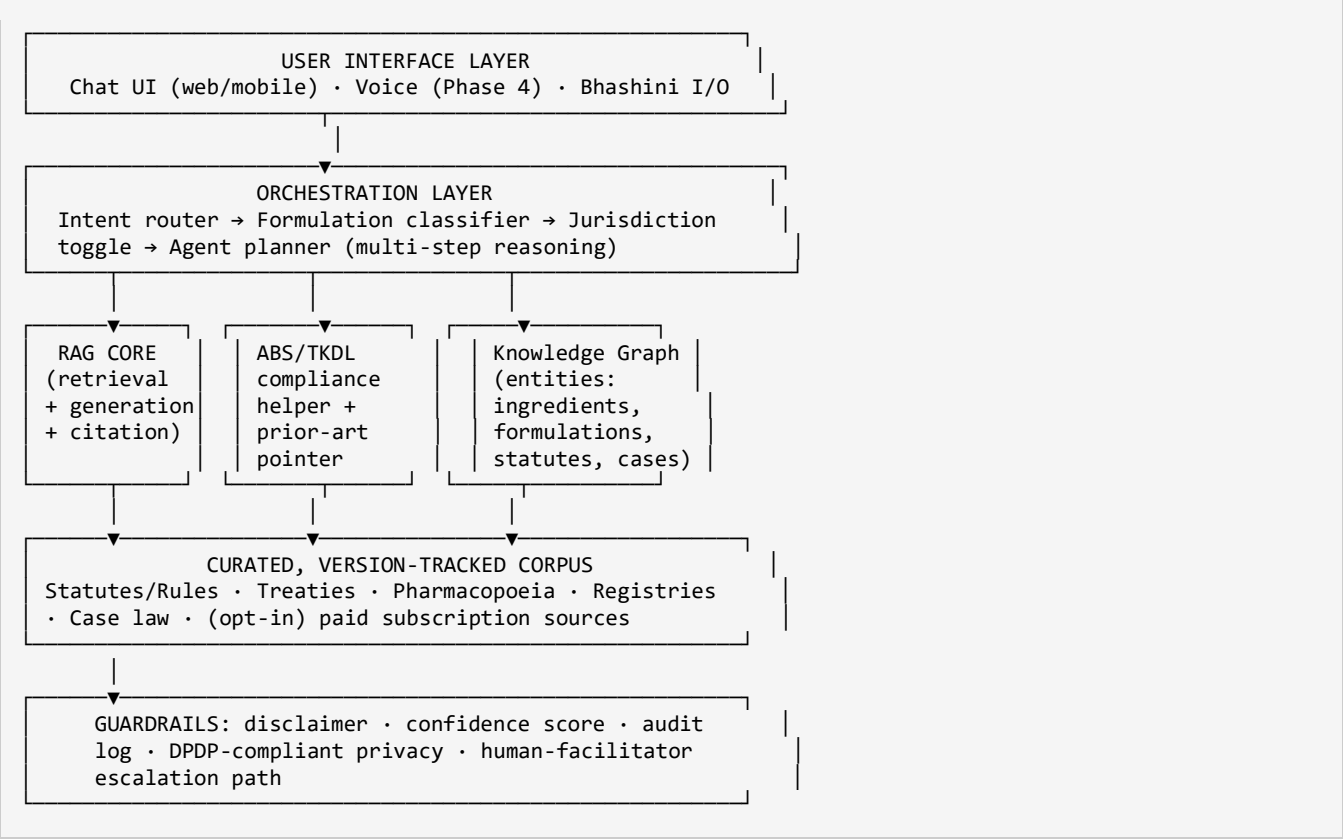
Ayurlex answers Ayurveda-specific IP and regulatory questions with source-grounded, jurisdiction-clear, non-hallucinated answers — bridging the gap between AYUSH practitioners/startups and the overlapping worlds of patents, GI, trademarks, ABS compliance, and drug regulation.

Design tenets (non-negotiable)

- Every claim traceable to a statute/rule/treaty/record.
- India vs. International answers are never blended in one response.
- "Information, not legal advice" disclaimer on every session.
- Abstain rather than guess when confidence is low.

2. High-Level Architecture

The system is organized into five layers: user interface, orchestration, three parallel reasoning engines (RAG core, ABS/TKDL helper, knowledge graph), a curated corpus, and a guardrails layer that wraps every response.



3. Core Modules

3.1 Formulation Classification Flow

A short decision-tree dialogue (≤5 questions) determines the product bucket, since IP/ABS posture differs completely by type, as the first gate for every product query:

Bucket	Signal questions	IP/ABS implication
Classical / generic medicine	Formulation + method in First-Schedule text?	TK; Section 3(p) patent bar; defend via TKDL
Patent / proprietary medicine	Modified classical formulation, own brand?	Trademark + limited patent scope
New / non-classical drug	Novel combination, no textual precedent?	Genuine patent potential; needs safety/efficacy data
Phytopharmaceutical	Standardized extract, CDSCO pathway?	Patent + regulatory dossier
Ayurveda-Aahar / nutraceutical	Food-with-health-claim positioning?	FSSAI regime, not drug IP
Cosmetic	Topical, cosmetic claims only?	Cosmetic rules, design/trademark focus

Output: a routing tag that conditions all downstream retrieval and disclaimers.

3.2 Jurisdiction Toggle

- Hard UI/state switch: India ⟷ International.
- Each retrieval call is scoped to one jurisdiction's document set; the generator is prompted never to merge them in a single answer unless the user explicitly asks for a comparison — in which case output is rendered as two clearly labeled columns/sections, not fused prose.

3.3 RAG Core

- Chunking: statute/section-aware (not fixed-token) — one chunk ≈ one section/rule/article, with parent-doc metadata (Act, year, amendment status).
- Embedding + retrieval: hybrid dense (multilingual embedding model) + sparse (BM25) for exact statutory phrase matches; re-ranker before generation.
- Generation: constrained to cite retrieved chunk IDs; refuses to answer beyond retrieved context (no parametric-memory legal claims).
- Confidence indicator: derived from retrieval score spread + citation coverage of the answer; low confidence → abstain + escalate.

3.4 ABS Compliance Helper + TKDL / Prior-Art Pointer

- Given a biological resource + intended use, walks through Biological Diversity Act (2023 amendment) + 2024 Rules obligations, flags whether NBA/SBB approval is triggered.
- TKDL search wrapper: takes formulation description → returns likely-relevant TKDL classification codes/entries as a prior-art starting point (not a legal clearance).

3.5 Knowledge Graph (Phase 2+)

Entities: ingredients ↔ classical texts ↔ formulations ↔ patents/GI/TM filings ↔ statutes ↔ case law. Enables multi-hop questions like "which GI-tagged formulations use ingredient X, and has X appeared in a rejected 3(p) patent?"

3.6 Agentic Orchestration (Phase 2+)

Planner agent decomposes multi-part queries into sub-tasks (classify → retrieve national → retrieve international → cross-check TKDL → synthesize), calling RAG core and KG as tools, with a trace log for auditability.

3.7 Multilingual Layer

- Bhashini APIs for ASR/TTS + translation at the UI edge.
- Retrieval/generation core stays in a pivot language (English or Hindi); translate in/out at the boundary to keep the curated corpus single-sourced and avoid translation drift in legal text.

3.8 Guardrails & Compliance

- Standing disclaimer banner + repeated in every substantive answer.
- Escalation button → human IP facilitator handoff with conversation summary.
- DPDP Act alignment: consent capture, data minimization, logged/explicit permission before using a user's paid-subscription sources, audit trail of retrieval + citations.
- Never fabricate a citation — if no chunk supports a claim, the answer must say so.

4. Tech Stack (Suggested)

Layer	Options
LLM	Claude (Anthropic API) — strong multilingual + citation discipline
Embeddings	Multilingual model (e.g., BGE-M3 / Voyage multilingual)
Vector DB	Qdrant / Weaviate / pgvector
Sparse search	Elasticsearch / OpenSearch (BM25)
Knowledge Graph	Neo4j
Orchestration	LangGraph / custom agent loop
Backend	FastAPI
Frontend	React + i18n
Voice / translation	Bhashini APIs
Auth / Data	DPDP-compliant cloud (data residency in India)

5. Data Pipeline

- Ingest: scrape/download from TKDL, India Code, IP India (Patents/InPASS, TM, Design, GI Registry), NBA/ABS portal, CDSCO, FSSAI, WIPO/WTO (TRIPS), CBD/Nagoya texts, PCT/Madrid/Hague/Budapest docs.
- Normalize: OCR where needed, structure into Act → Chapter → Section, tag with jurisdiction + effective date + amendment lineage.
- Version-track: every document has an ingestion date + source URL + checksum; superseded versions archived, not deleted, so historical queries remain answerable.
- Chunk + embed per Section 3.3.
- Human review pass on high-stakes categories (patent bar, ABS triggers) before go-live.

6. Staged Build Roadmap

Phase	Scope
MVP	Citation-grounded retrieval + generation over national + international statutory corpus; jurisdiction toggle; formulation classifier; basic confidence indicator; disclaimer; English + Hindi
Phase 2	Knowledge graph; agentic multi-step orchestration; ABS helper + TKDL pointer; human-facilitator escalation
Phase 3	Paid-source connectors (with explicit logged user consent)
Phase 4	Full multilingual (Bhashini) + voice experience; case-law layer

7. Evaluation Framework

- Answer accuracy: expert-graded against gold Q&A set per category (classical / new drug / GI / etc.).
- Citation correctness: precision/recall of cited sections against ground-truth relevant sections.
- Safe abstention: rate of correct "insufficient information" responses on out-of-scope/adversarial queries.
- Jurisdiction separation: automated check that no single answer mixes India + international citations without explicit comparison request.
- Multilingual quality: back-translation BLEU/COMET + native-speaker review for legal-term fidelity.

8. Key Risks to Design Against

Risk	Mitigation
Hallucinated citations	Hard constraint: generation must quote only from retrieved chunk IDs; post-hoc citation verifier step.

Risk	Mitigation
Jurisdiction bleed	Separate retrieval indices per jurisdiction, not just prompt instructions.
Stale law	Version-tracked corpus + scheduled re-crawl + "as of" date stamped on every answer.
Users treating output as legal advice	Disclaimer + escalation path, reinforced whenever the query pattern looks like a filing decision.

9. Implementation Architecture — Frontend & Backend

Frontend and backend are split as independent deployables communicating over a versioned REST/streaming API — no shared runtime, so either can be rebuilt or scaled on its own.

9.1 Request Flow (End to End)

```
User query (+ jurisdiction, lang)
        │
        ▼
Frontend → POST /api/v1/chat/query  (SSE/stream response)
        │
        ▼
API Gateway (auth, rate-limit, DPDP consent check)
        │
        ▼
Orchestration Service
   ├─ Formulation Classifier (if unset for session)
   ├─ Jurisdiction Router (India | International index)
   ├─ RAG Service ──► Vector DB + Sparse Index ──► Re-ranker
   ├─ ABS/TKDL Service (conditional)
   ├─ Knowledge Graph Service (conditional, Phase 2+)
   └─ LLM Generation (Claude) — citation-constrained
        │
        ▼
Citation Verifier → Confidence Scorer → Disclaimer Injector
        │
        ▼
Response streamed back to Frontend (answer + citations + confidence + escalation CTA)
        │
        ▼
Audit Logger (async, write-behind) → DPDP-compliant store
```

9.2 Frontend Architecture

Stack: React + TypeScript, Vite, TanStack Query (server state), Zustand/Redux (UI state), i18next (multilingual UI strings, separate from corpus language), Tailwind or design-system CSS.

Folder structure

```
frontend/
├── public/
├── src/
│   ├── app/                    # app shell, routing, providers
│   │   ├── App.tsx
│   │   ├── router.tsx
│   │   └── providers/          # QueryClient, i18n, Auth, Theme
│   ├── features/
│   │   ├── chat/
│   │   │   ├── components/     # MessageBubble, CitationChip, ConfidenceBadge
│   │   │   ├── hooks/          # useChatStream, useSession
│   │   │   ├── api.ts          # calls /chat/query (SSE client)
│   │   │   └── store.ts
│   │   ├── jurisdiction-toggle/
│   │   │   ├── components/     # India/International switch, comparison view
│   │   │   └── store.ts
│   │   ├── formulation-classifier/
│   │   │   ├── components/     # decision-tree question flow
│   │   │   └── api.ts
│   │   ├── abs-tkdl/
```

```
│   │   │   └── components/     # ABS wizard, TKDL result cards
│   │   ├── escalation/
│   │   │   └── components/     # "Talk to an IP facilitator" flow
│   │   └── voice-multilingual/ # Bhashini widgets (Phase 4)
│   ├── shared/
│   │   ├── ui/                 # buttons, modals, disclaimer banner
│   │   ├── api-client/         # axios/fetch wrapper, auth interceptor
│   │   ├── i18n/                # locale JSON files
│   │   └── types/               # shared TS interfaces (Citation, Answer, etc.)
│   ├── pages/                   # Chat, History, Admin (corpus status), Settings
│   └── main.tsx
├── tests/
└── package.json
```

Key frontend responsibilities: render streamed answers incrementally; keep jurisdiction state pinned across a session; render citations as clickable chips linking to source registry/statute; surface confidence + abstention states distinctly from normal answers; persist consent flags before enabling paid-source usage; never let a UI bug silently drop the disclaimer.

9.3 Backend Architecture

Stack: Python, FastAPI (async), Pydantic schemas, Celery/RQ for async ingestion jobs, PostgreSQL (relational: users, sessions, audit, consent), pgvector or Qdrant (embeddings), OpenSearch (sparse/BM25), Neo4j (knowledge graph, Phase 2+), Redis (cache/session), Claude API (generation).

Folder structure

```
backend/
├── app/
│   ├── main.py                      # FastAPI app entrypoint
│   ├── api/
│   │   └── v1/
│   │       ├── chat.py              # /chat/query (streaming)
│   │       ├── classifier.py        # /classify/formulation
│   │       ├── abs.py               # /abs/check, /tkdl/search
│   │       ├── jurisdiction.py      # /jurisdiction/compare
│   │       ├── escalation.py        # /escalate
│   │       └── admin.py             # corpus status, ingestion triggers
│   ├── orchestration/
│   │   ├── planner.py               # agentic task decomposition
│   │   ├── router.py                # jurisdiction + intent routing
│   │   └── pipeline.py              # orchestrates RAG→verify→score→respond
│   ├── rag/
│   │   ├── retriever.py             # hybrid dense+sparse retrieval
│   │   ├── reranker.py
│   │   ├── chunker.py               # statute/section-aware chunking
│   │   ├── citation_verifier.py
│   │   └── confidence_scorer.py
│   ├── knowledge_graph/
│   │   ├── schema.py                # node/edge definitions
│   │   └── queries.py               # multi-hop Cypher queries
│   ├── abs_tkdl/
│   │   ├── abs_rules_engine.py      # BDA 2023 + 2024 Rules logic
│   │   └── tkdl_client.py
│   ├── llm/
│   │   ├── claude_client.py
│   │   └── prompts/                 # versioned system prompts per module
│   ├── ingestion/
│   │   ├── sources/                 # one scraper/loader per source (TKDL,
│   │   │                            #   IndiaCode, IPIndia, NBA, CDSCO, FSSAI, WIPO...)
```

```
│   │   ├── normalizer.py             # OCR, structuring, tagging
│   │   ├── versioning.py             # checksum, supersession tracking
│   │   └── jobs.py                   # scheduled re-crawl tasks (Celery)
│   ├── data/
│   │   ├── models/                   # SQLAlchemy models: User, Session,
│   │   │                             #   Query, Citation, ConsentLog
│   │   ├── repositories/
│   │   └── migrations/               # Alembic
│   ├── security/
│   │   ├── auth.py                   # OAuth2/JWT
│   │   ├── consent.py                # DPDP consent capture/check
│   │   └── audit_logger.py
│   ├── config.py
│   └── deps.py                       # DI: db session, vector client, etc.
├── tests/
│   ├── unit/
│   ├── integration/
│   └── eval/                         # accuracy/citation/abstention harness
├── scripts/                           # one-off corpus bootstrap scripts
├── alembic.ini
└── requirements.txt / pyproject.toml
```

Key backend responsibilities: enforce jurisdiction separation at the retrieval-index level (not just prompting); block generation if citation coverage falls under threshold (return abstention instead); write an immutable audit record per query (sources retrieved, citations returned, confidence, jurisdiction); gate paid-source retrieval behind an explicit per-user consent flag; expose an admin surface for corpus version/freshness status.

9.4 API Contract (Representative)

Endpoint	Method	Purpose
/api/v1/chat/query	POST (stream)	Main Q&A, takes {query, jurisdiction, lang, session_id}
/api/v1/classify/formulation	POST	Runs/continues the classification decision tree
/api/v1/jurisdiction/compare	POST	Explicit side-by-side India vs. International
/api/v1/abs/check	POST	ABS trigger assessment for a resource + use
/api/v1/tkdl/search	POST	Prior-art pointer lookup
/api/v1/escalate	POST	Hands off session to human IP facilitator
/api/v1/admin/corpus/status	GET	Ingestion freshness, source health

9.5 Deployment Split

- Frontend: static build (Vite) on a CDN/edge host; talks only to the API gateway, never directly to DB/vector store.
- Backend: containerized (Docker), API service scaled independently from the ingestion-worker pool (Celery workers), since crawling/OCR is bursty and query traffic is steady.
- Data stores: Postgres (managed, India region for DPDP), vector DB and OpenSearch as separate stateful services, Neo4j added in Phase 2 without touching the API contract (new internal service, same /chat/query surface).